

Java: Matriz

Matrizes

Prof. Dionizio Porfírio
Unichristus • 2026.2

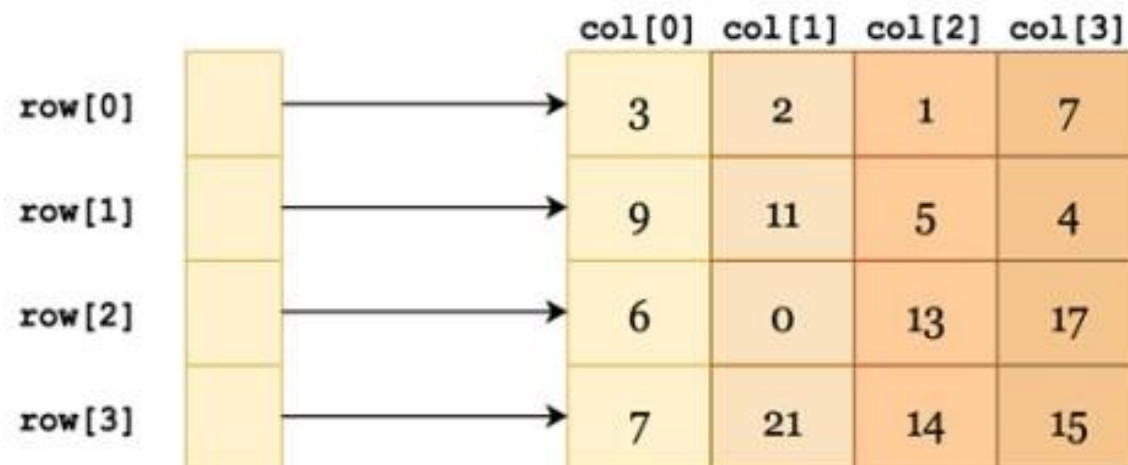
Objetivos

Ao final da aula, o você deverá ser capaz de:

- Compreender o conceito de matriz como um conjunto de variáveis do mesmo tipo, organizadas em duas ou mais dimensões.
- Diferenciar matrizes bidimensionais de vetores e reconhecer matrizes multidimensionais.
- Declarar uma matriz em Java.
- Definir o tamanho de cada dimensão da matriz.
- Identificar as posições de uma matriz por meio de índices.
- Atribuir valores aos elementos de uma matriz.
- Acessar e apresentar os valores armazenados em suas posições.

Definição de matriz

Uma matriz é formada por um conjunto de variáveis do mesmo tipo, identificadas por um único nome. Cada variável é localizada por meio de sua posição na estrutura. Em Java, podem ser declaradas matrizes unidimensionais, também chamadas de vetores, além de matrizes bidimensionais e multidimensionais. As matrizes de duas dimensões são as mais utilizadas e exigem um índice para cada dimensão. Os índices começam em 0 e vão até o tamanho da dimensão menos 1, devendo ser representados por algum dos tipos inteiros disponíveis na linguagem.



The diagram illustrates a 4x4 matrix. On the left, a vertical column of four yellow boxes represents the rows, labeled row[0], row[1], row[2], and row[3]. Arrows point from each row label to the corresponding row in the matrix. Above the matrix, a horizontal row of four labels represents the columns: col[0], col[1], col[2], and col[3]. The matrix itself is a 4x4 grid of cells, each containing a number. The cells are colored in a gradient from light yellow to orange. The values in the matrix are:

	col[0]	col[1]	col[2]	col[3]
row[0]	3	2	1	7
row[1]	9	11	5	4
row[2]	6	0	13	17
row[3]	7	21	14	15

Declaração de matriz

Em Java, uma matriz é declarada utilizando colchetes vazios após o tipo de dado ou após o nome da variável.

Em seguida, é necessário definir o tamanho de cada dimensão.

Esse tamanho pode ser informado por um valor inteiro fixo, chamado de literal ou constante, ou por uma variável cujo valor seja definido durante a execução do programa.

Para utilizar uma matriz, são necessários dois passos:

1º passo — Declarar a variável

```
tipo_dos_dados nome_variavel[][][]...[];
```

Os pares de colchetes indicam a quantidade de dimensões da matriz.

2º passo — Definir o tamanho das dimensões

```
nome_variavel = new tipo_dos_dados[dimensao1][dimensao2]...[dimensaoN];
```

Nesse comando:

- tipo_dos_dados indica o tipo de valores armazenados;
- nome_variavel é o identificador da matriz;
- cada dimensão indica o tamanho correspondente daquela dimensão.

Exemplos de matriz

Nos exemplos a seguir, são utilizadas duas linhas de comando: a primeira declara a matriz, enquanto a segunda define o tamanho de suas dimensões. É importante destacar que, em Java, os pares de colchetes podem ser colocados antes ou depois do nome da variável, ou distribuídos entre essas duas posições. Portanto, todas as formas apresentadas a seguir são válidas.

Exemplo 1:

```
float X[][];  
X = new float[2][6];
```

A declaração anterior criou uma variável chamada X contendo duas linhas (0 a 1) com seis colunas cada (0 a 5), capazes de armazenar números reais, como pode ser observado a seguir.

		0	1	2	3	4	5
X	0						
	1						

Exemplos de matriz

Exemplo 2:

```
char [][]MAT;  
MAT = new char[4][3];
```

A declaração anterior criou uma variável chamada MAT contendo quatro linhas (0 a 3) com três colunas cada (0 a 2), capazes de armazenar caracteres, como pode ser observado a seguir.

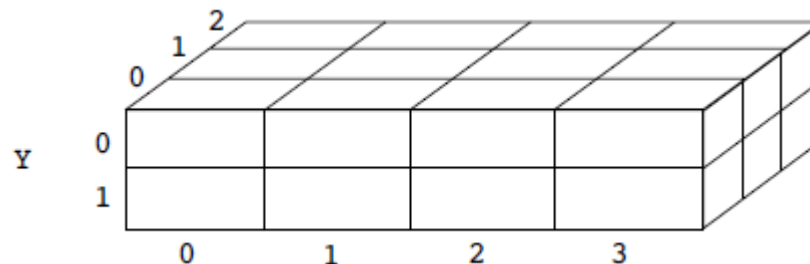
		0	1	2
MAT	0			
	1			
	2			
	3			

Exemplos de matriz

Exemplo 3:

```
int [][]Y[];  
Y = new int[2][4][3];
```

A declaração anterior criou uma variável chamada Y contendo duas linhas (0 a 1) com quatro colunas (0 a 3) e três profundidades (0 a 2), capazes de armazenar números inteiros, como pode ser observado a seguir.



Além das formas apresentadas nos exemplos anteriores, a linguagem Java também permite realizar os dois passos necessários para a utilização de uma matriz — declaração e dimensionamento — em uma única linha de comando. Essa forma condensada de criação permite declarar a matriz e definir o tamanho de suas dimensões simultaneamente.

Exemplos de matriz

Exemplo 4:

```
float X[][] = new float[2][6];
```

A declaração anterior criou uma variável chamada X contendo duas linhas (0 a 1) com seis colunas cada (0 a 5), capazes de armazenar números reais, como pode ser observado a seguir.

		0	1	2	3	4	5
X	0						
	1						

Exemplos de matriz

Exemplo 5:

```
char [][]MAT = new char[4][3];
```

A declaração anterior criou uma variável chamada MAT contendo quatro linhas (0 a 3) com três colunas cada (0 a 2), capazes de armazenar caracteres, como pode ser observado a seguir.

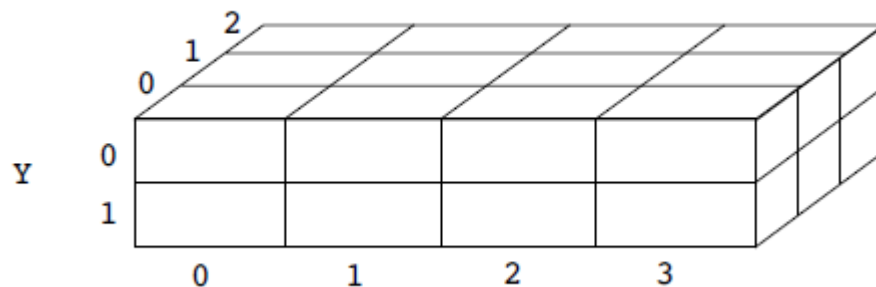
		0	1	2
MAT	0			
	1			
	2			
	3			

Exemplos de matriz

Exemplo 6:

```
int [][]Y[] = new int[2][4][3];
```

A declaração anterior criou uma variável chamada Y contendo duas linhas (0 a 1) com quatro colunas (0 a 3) e três profundidades (0 a 2), capazes de armazenar números inteiros, como pode ser observado a seguir.



É importante lembrar, também, que o tamanho das dimensões não precisa ser feito em um mesmo momento. Os exemplos que seguem mostram essa flexibilidade.

Exemplos de matriz

Exemplo 7:

Nesse exemplo, cuja numeração das linhas foi utilizada apenas para facilitar a explicação, a linha 1 define a variável `Y` como uma matriz bidimensional. A linha 2 estabelece que a primeira dimensão possui tamanho 2, ou seja, a matriz terá duas linhas, identificadas pelos índices 0 e 1. Na linha 3, é definido o tamanho da linha 0, que passa a possuir cinco colunas. Já a linha 4 define o tamanho da linha 1, que passa a possuir duas colunas.

```
1. int Y[][];  
2. Y = new int[2][];  
3. Y[0] = new int[5];  
4. Y[1] = new int[2];
```

Uma representação dessa matriz poderia ser:

Matriz Y	0					
	1					
		0	1	2	3	4

Atribuindo valores a uma matriz

Atribuir valor a uma matriz significa armazenar uma informação em um de seus elementos, identificado de forma única por meio de seus índices.

`X[1][4]=5;` → Atribui o valor 5 à posição identificada pelos índices 1 (2ª linha) e 4 (5ª coluna).

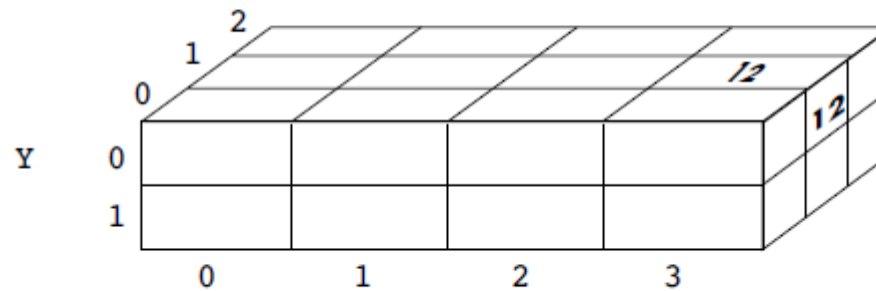
		0	1	2	3	4	5
X	0						
	1					5	

`MAT[3][2] = 'D';` → Atribui a letra D à posição identificada pelos índices 3 (4ª linha) e 2 (3ª coluna).

		0	1	2
MAT	0			
	1			
	2			
	3			D

Atribuindo valores a uma matriz

$Y[0][3][1] = 12;$ → Atribui o valor 12 à posição identificada pelos índices 0 (1ª linha), 3 (4ª coluna) e 1 (2ª profundidade).



Preenchendo uma matriz

Preencher uma matriz significa percorrer todos os seus elementos, atribuindo-lhes um valor. Esse valor pode ser recebido do usuário, por meio do teclado, ou pode ser gerado pelo programa.

Exemplo 1:

```
int X[][] = new int[7][3];
Scanner e = new Scanner(System.in);
for (i=0;i<7;i++)
{
    for (j=0;j<3;j++)
        X[i][j] = e.nextInt();
}
```

Como a matriz possui 7 linhas e 3 colunas, foram utilizadas duas estruturas de repetição for. A variável i percorre as linhas, assumindo valores de 0 a 6, enquanto a variável j percorre as colunas, assumindo valores de 0 a 2. Dessa forma, cada posição da matriz X é preenchida com um valor digitado pelo usuário por meio do método nextInt() da classe Scanner.

Preenchendo uma matriz

Exemplo 2:

```
1. int MAT[][];  
2. MAT = new int[3][];  
3. MAT[0]= new int[2];  
4. MAT[1]= new int[5];  
5. MAT[2]= new int[3];  
6. Scanner e = new Scanner(System.in);  
7. for (i=0;i<MAT.length;i++)  
8. {  
9.     for (j=0;j<MAT[i].length;j++)  
10.         X[i][j] = e.nextInt();  
11. }
```

No exemplo 2, cuja numeração foi utilizada apenas para facilitar a explicação, é apresentada uma matriz cujas linhas possuem tamanhos diferentes. A linha 1 define MAT como uma matriz bidimensional. A linha 2 estabelece que a matriz terá três linhas, sem definir ainda a quantidade de colunas. Em seguida, as linhas 3, 4 e 5 determinam, respectivamente, que as linhas 0, 1 e 2 terão 2, 5 e 3 colunas.

Essa diferença no tamanho das linhas exige um recurso que permita consultar, quando necessário, o tamanho de cada dimensão de um array, seja ele um vetor ou uma matriz. Em Java, esse recurso é o atributo `length`. Na linha 7, `MAT.length` representa o tamanho da primeira dimensão de MAT, ou seja, a quantidade de linhas, que é igual a 3. Já na linha 9, `MAT[i].length` representa o tamanho da linha indicada por `i`: quando `i` vale 0, o tamanho é 2; quando vale 1, é 5; e quando vale 2, é 3.

Preenchendo uma matriz

Exemplo 2:

```
1. int MAT[][];  
2. MAT = new int[3][];  
3. MAT[0]= new int[2];  
4. MAT[1]= new int[5];  
5. MAT[2]= new int[3];  
6. Scanner e = new Scanner(System.in);  
7. for (i=0;i<MAT.length;i++)  
8. {  
9.     for (j=0;j<MAT[i].length;j++)  
10.         X[i][j] = e.nextInt();  
11. }
```

As duas estruturas de repetição for garantem que i percorra todas as linhas, de 0 a 2, enquanto j percorre todas as colunas existentes em cada linha, respeitando seus respectivos tamanhos. Assim, a cada execução das estruturas de repetição, uma posição diferente da matriz é preenchida com um valor digitado pelo usuário por meio do método nextInt() da classe Scanner.

Mostrando os elementos de uma matriz

Pode-se, também, percorrer todos os elementos da matriz, acessando seu conteúdo.

Exemplo 1:

```
for (i=0; i<10; i++)  
{  
    for (j=0; j<6; j++)  
        System.out.println(X[i][j]);  
}
```

No exemplo 1 foram mostrados todos os elementos de uma matriz contendo dez linhas e seis colunas. Observe que são usados dois índices, i e j. Esses índices estão atrelados a estruturas de repetição que mantêm a variação de ambos dentro de intervalos permitidos. Ou seja, o índice i, que representa as linhas, varia de 0 a 9 e o índice j, que representa as colunas, varia de 0 a 5.

Mostrando os elementos de uma matriz

Exemplo 2:

```
1. for (i=0;i<MAT.length;i++)  
2. {  
3.     for (j=0;j<MAT[i].length;j++)  
4.         System.out.println(MAT[i][j]);  
5. }
```

No exemplo 2, também foram apresentados todos os elementos de uma matriz. Entretanto, devido ao uso do atributo `length`, apresentado em detalhes na Seção 7.4.5, as linhas dessa matriz podem possuir tamanhos diferentes.

Na linha 1, observa-se o uso de `MAT.length`, que representa o tamanho da primeira dimensão de `MAT`, ou seja, a quantidade de linhas. Já na linha 3, `MAT[i].length` representa o tamanho da linha `i` da matriz.

As duas estruturas de repetição `for` garantem que a variável `i` percorra todas as linhas e que a variável `j` percorra todas as colunas, respeitando os dimensionamentos definidos. Assim, a cada execução das estruturas de repetição, uma posição diferente da matriz é acessada, e seu valor é apresentado por meio do método `println()` da classe `System`.

Percorrendo uma matriz

Anteriormente, nos tópicos 7.4.5 e 7.4.6, foram apresentadas formas de preencher uma matriz e mostrar todas as suas posições. Nessas operações, foi necessário percorrer todos os elementos da matriz. Uma das maneiras mais simples de realizar esse percurso é utilizar uma estrutura de repetição para cada dimensão. A ordem dessas estruturas define a forma como a matriz será percorrida. A seguir, serão apresentadas duas formas de percorrer a mesma matriz x, composta por 3 linhas e 4 colunas.

Matriz x	0	4	5	1	10
	1	16	11	76	8
	2	9	54	32	89
		0	1	2	3

Percorrendo uma matriz

Forma 1: precisamos percorrer a matriz, de tal forma que seja possível mostrar todos os elementos gravados em cada linha. Para isso, utilizaremos duas estruturas de repetição, conforme mostrado a seguir (a numeração à esquerda não faz parte do programa, servirá apenas para facilitar a explicação).

```
for(i=0;i<3;i++)
{
    System.out.println("Elementos da linha " + i);
    for(j=0;j<4;j++)
    {
        System.out.println(x[i,j]);
    }
}
```

Percorrendo uma matriz

A primeira estrutura de repetição, na linha 1, é controlada pela variável i , que assume valores de 0 a 2. A cada execução dessa estrutura, é iniciada a segunda estrutura de repetição, na linha 4, controlada pela variável j , que assume valores de 0 a 3. Dessa forma, cada valor de i é associado a quatro valores de j .

Esse arranjo permite apresentar os elementos separados por linhas. Enquanto i permanece fixo, j percorre os valores de 0 a 3, formando todos os pares possíveis de índices. Quando i vale 0, são apresentados os elementos da primeira linha: $x[0][0]$, $x[0][1]$, $x[0][2]$ e $x[0][3]$. Em seguida, i passa a valer 1, enquanto j percorre novamente os valores de 0 a 3, permitindo acessar toda a segunda linha: $x[1][0]$, $x[1][1]$, $x[1][2]$ e $x[1][3]$. O mesmo procedimento continua para os demais valores de i .

Matriz x

0	4	5	1	10
1	16	11	76	8
2	9	54	32	89
	0	1	2	3

Percorrendo uma matriz

Forma 2: precisamos percorrer a matriz, de tal forma que seja possível mostrar todos os elementos gravados em cada coluna. Para isso, utilizaremos duas estruturas de repetição, conforme mostrado a seguir (a numeração à esquerda não faz parte do programa, servirá apenas para facilitar a explicação).

```
for (i=0;i<4;i++)
{
    System.out.println("Elementos da coluna " + i);
    for (j=0;j<3;j++)
    {
        System.out.println(x[j,i]);
    }
}
```

Percorrendo uma matriz

Matriz x

0	4	5	1	10
1	16	11	76	8
2	9	54	32	89
	0	1	2	3

A primeira estrutura de repetição, controlada pela variável i , assume valores de 0 a 3. A cada execução, é iniciada a segunda estrutura, controlada pela variável j , que assume valores de 0 a 2. Assim, cada valor de i é associado a três valores de j , formando todos os pares possíveis de índices. Quando i vale 0, são apresentados os elementos da primeira coluna: $x[0][0]$, $x[1][0]$ e $x[2][0]$. Em seguida, i passa a valer 1, enquanto j percorre novamente os valores de 0 a 2, permitindo acessar a segunda coluna: $x[0][1]$, $x[1][1]$ e $x[2][1]$. Esse processo se repete para os demais valores de i , percorrendo a matriz coluna por coluna. A tabela apresenta uma simulação da execução do algoritmo, permitindo observar as alterações nos valores de i e j . A figura seguinte apresenta outra representação desse percurso, em que a direção das setas indica a mudança dessas variáveis e o caminho seguido na matriz.

Percorrendo uma matriz

Matriz x

0	4	5	1	10
1	16	11	76	8
2	9	54	32	89
	0	1	2	3

Nas formas de percurso apresentadas, a alteração dos valores de i e j permite formar todos os pares possíveis de linha e coluna da matriz. A variável i , utilizada no for externo, muda mais lentamente que j , utilizada no for interno. Por isso, i define o percurso realizado. No percurso horizontal, i varia de 0 a 2 e ocupa a primeira posição dentro dos colchetes. O índice da linha permanece fixo enquanto j percorre todas as colunas dessa linha. No percurso vertical, i varia de 0 a 3 e ocupa a segunda posição dentro dos colchetes. Nesse caso, o índice da coluna permanece fixo enquanto j percorre todas as linhas correspondentes àquela coluna.

Exercício

- 1.** Faça um programa que preencha uma matriz 3×5 com números inteiros, calcule e mostre a quantidade de elementos entre 15 e 20.

- 9.** Faça um programa que preencha uma matriz 3×3 com números reais e outro valor numérico digitado pelo usuário. O programa deverá calcular e mostrar a matriz resultante da multiplicação do número digitado por cada elemento da matriz.

- 14.** Faça um programa que preencha uma matriz 2×3 , calcule e mostre a quantidade de elementos da matriz que não pertencem ao intervalo $[5,15]$.

- 10.** Crie um programa que preencha uma matriz 5×5 com números inteiros, calcule e mostre a soma:
 - dos elementos da linha 4;
 - dos elementos da coluna 2;
 - dos elementos da diagonal principal;
 - dos elementos da diagonal secundária;
 - de todos os elementos da matriz.

Exercício

- 17.** Faça um programa que preencha e mostre a média dos elementos da diagonal principal de uma matriz 10×10 .
- 18.** Crie um programa que preencha uma matriz 5×5 de números reais, calcule e mostre a soma dos elementos da diagonal secundária.
- 23.** Faça um programa que preencha uma matriz 3×4 , calcule e mostre:
- a quantidade de elementos pares;
 - a soma dos elementos ímpares;
 - a média de todos os elementos.
- 3.** Elabore um programa que preencha uma matriz 6×3 , calcule e mostre:
- o maior elemento da matriz e sua respectiva posição, ou seja, linha e coluna;
 - o menor elemento da matriz e sua respectiva posição, ou seja, linha e coluna.

Exercício

21. Faça um programa que preencha uma matriz 5×5 de números reais. A seguir, o programa deverá multiplicar cada linha pelo elemento da diagonal principal daquela linha e mostrar a matriz após as multiplicações.

25. Crie um programa que:

- receba o preço de dez produtos e armazene-os em um vetor;
- receba a quantidade estocada de cada um desses produtos, em cinco armazéns diferentes, utilizando uma matriz 5×10 .

O programa deverá calcular e mostrar:

- a quantidade de produtos estocados em cada um dos armazéns;
- a quantidade de cada um dos produtos estocados, em todos os armazéns juntos;
- o preço do produto que possui maior estoque em um único armazém;
- o menor estoque armazenado;
- o custo de cada armazém.

Obrigado!